CS 301
High-Performance Computing

Lab Report-05

Mover Operation
Deferred and Immediate Insertion
Performance Analysis

Atik Vohra (202301447)
Soham Mevada (202301484)

Contents

1 Introduction **4**

2 Hardware Details **4**
2.1 LAB207 PCs 4
2.2 HPC Cluster 4

3 Mover Function with Insertion and deletion **5**
3.1 Mover function with deferred insertion 5
3.2 Mover function with immediate insertion 6
3.3 scalability of parallel mover function with deferred insertion 8
3.4 scalability of parallel mover function with immediate insertion 9

4 Answers to Analysis and Questions **10**

5 Conclusion **15**

1 Introduction

2 Hardware Details

2.1 LAB207 PCs

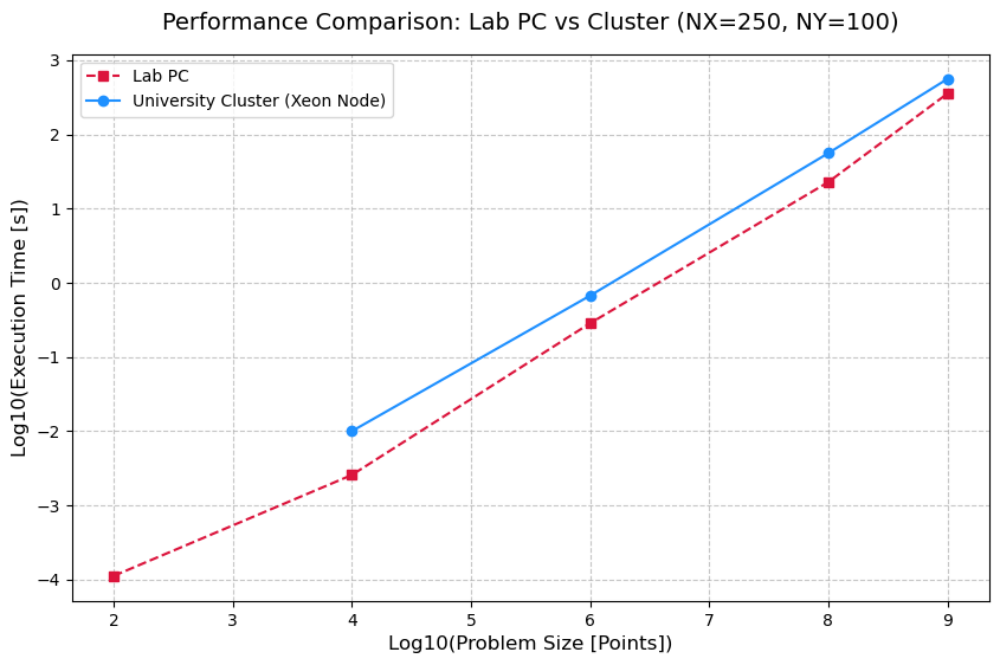
- Architecture: x86_64
- CPU(s): 12
- Threads per core: 2
- Cores per socket: 6
- Processor: Intel Core i5-12500
- L1 Cache: 288K
- L2 Cache: 7.5M
- L3 Cache: 18M

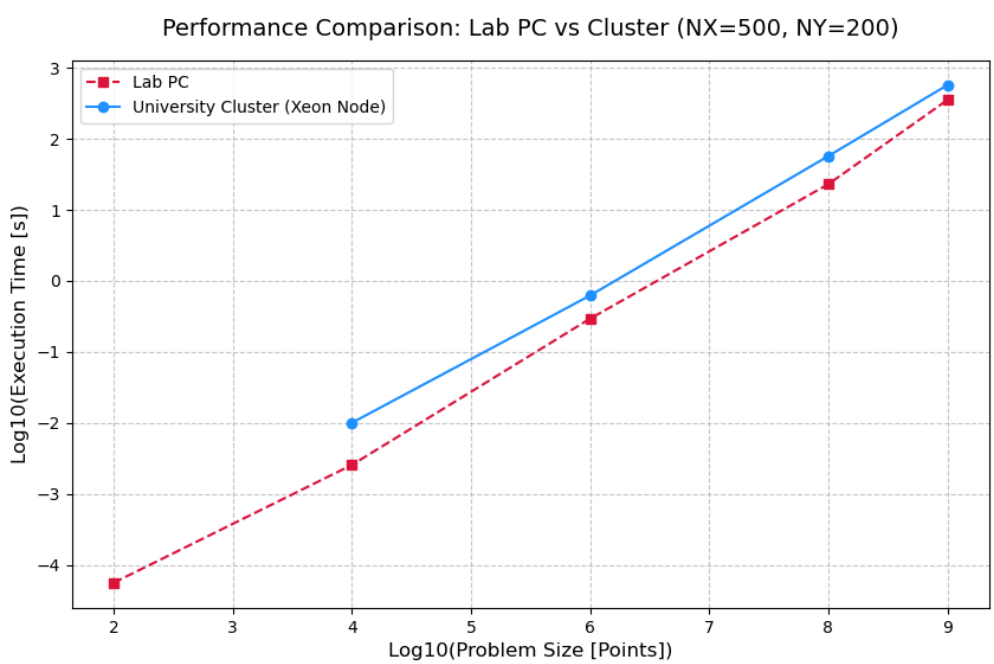
2.2 HPC Cluster

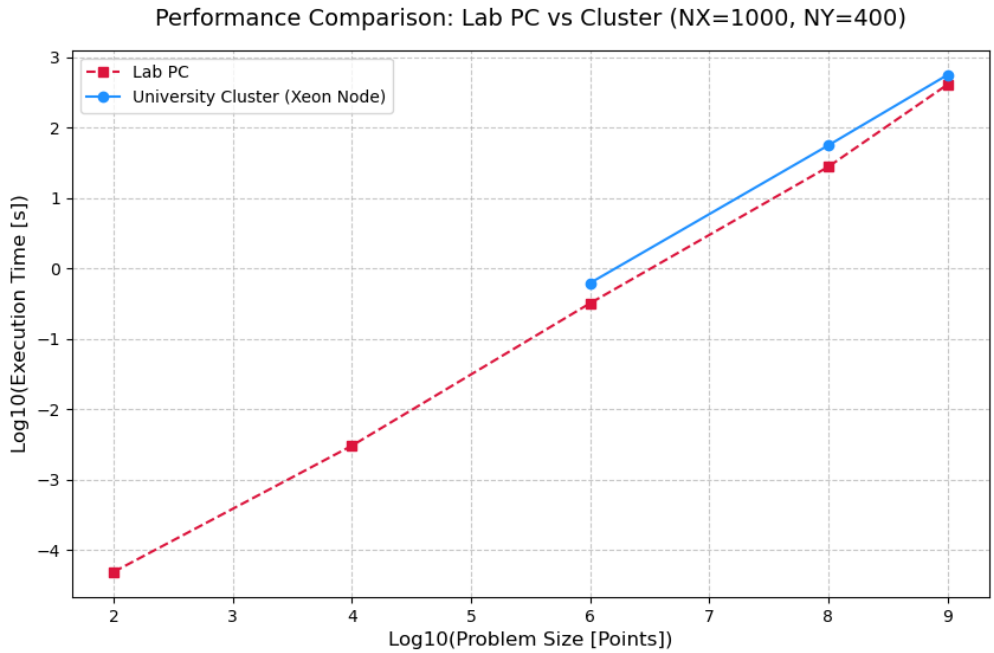
- Architecture: x86_64
- CPU(s): 16
- Threads per core: 1
- Cores per socket: 8
- Sockets: 2
- Processor: Intel Xeon E5-2640 v3
- L1 Cache: 32K
- L2 Cache: 256K
- L3 Cache: 20480K

3 Mover Function with Insertion and deletion

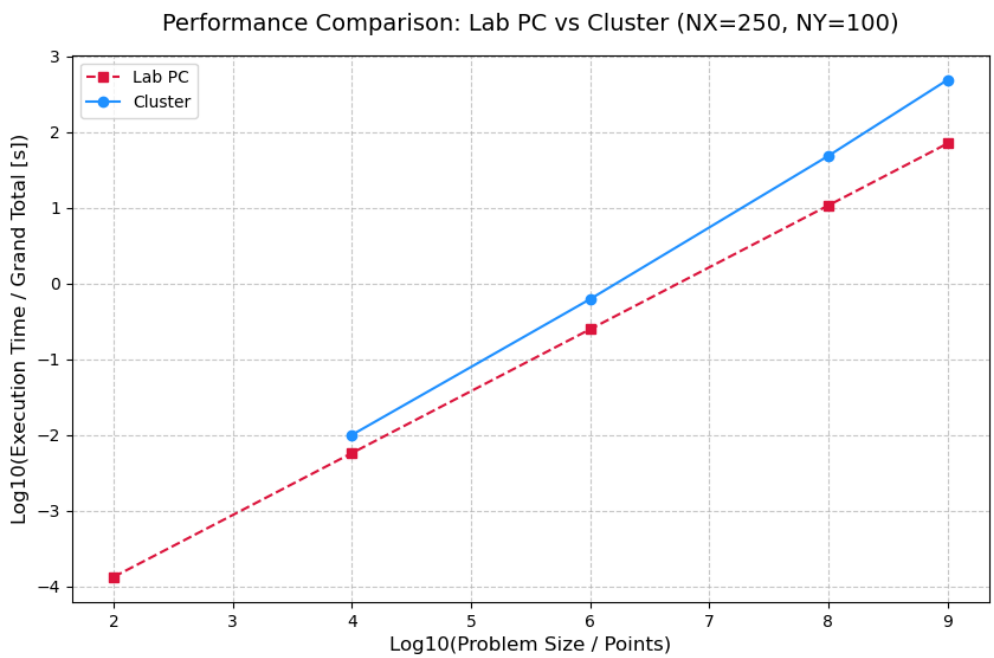
3.1 Mover function with deferred insertion

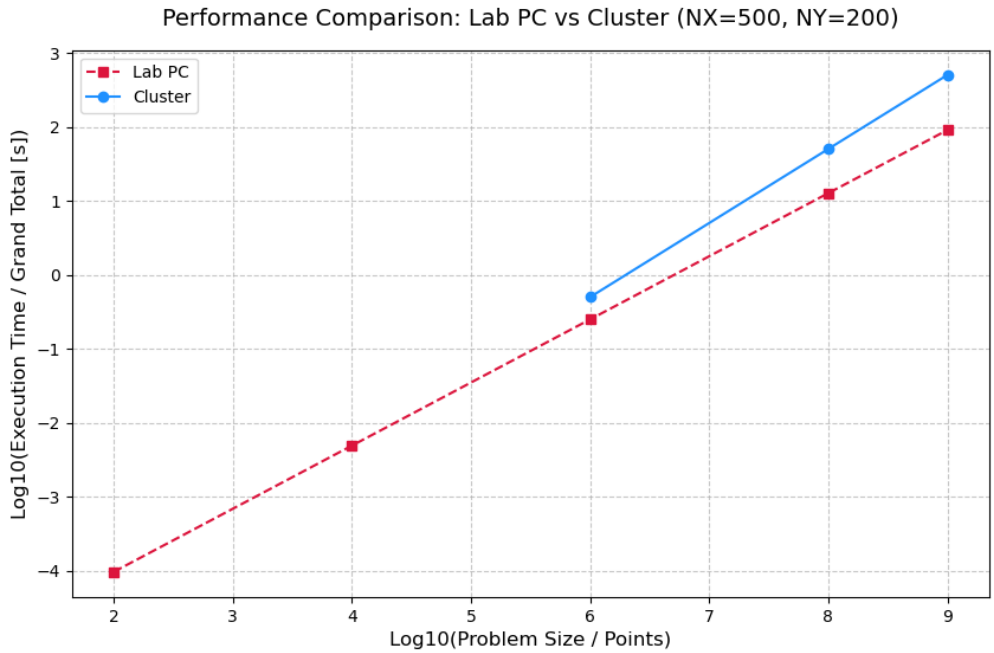


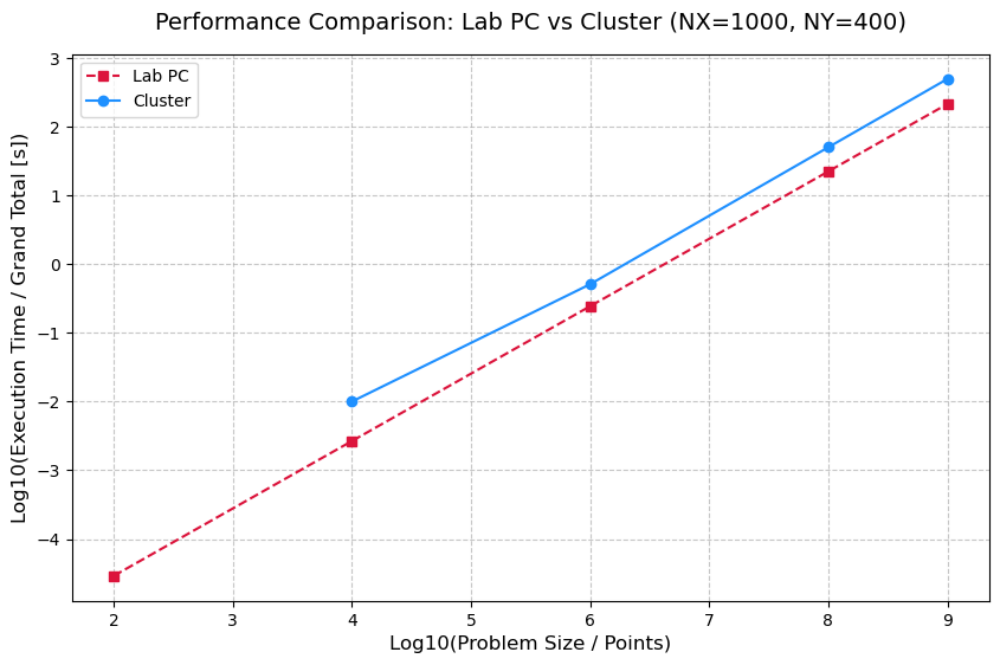




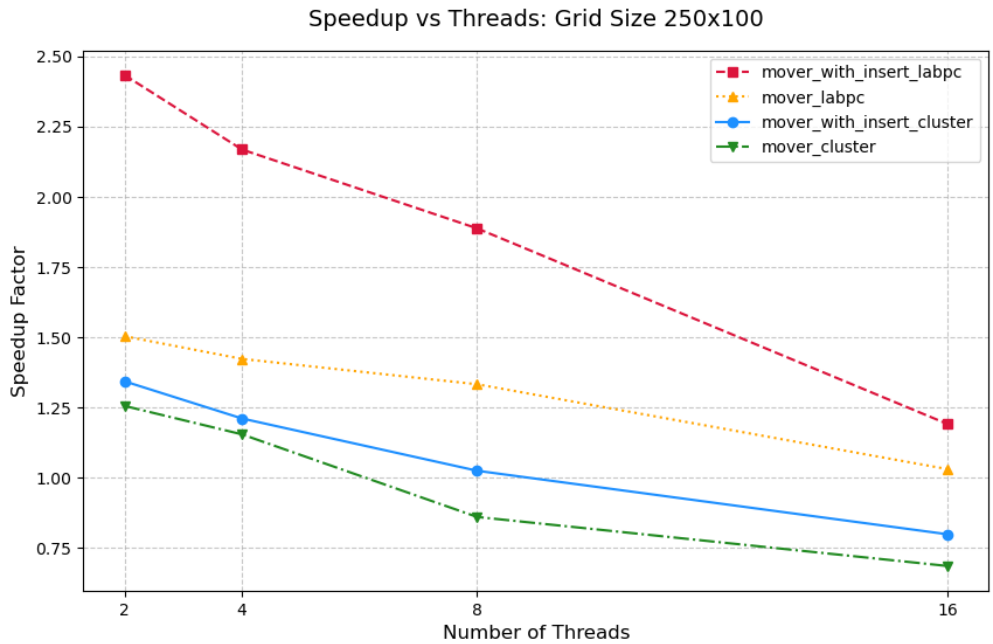
3.2 Mover function with immediate insertion

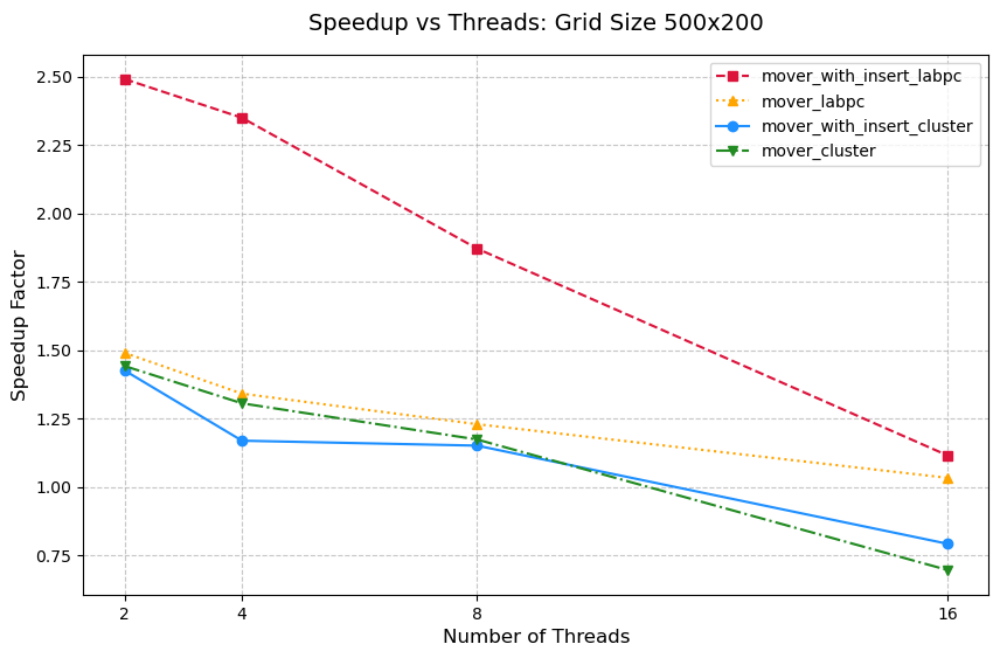


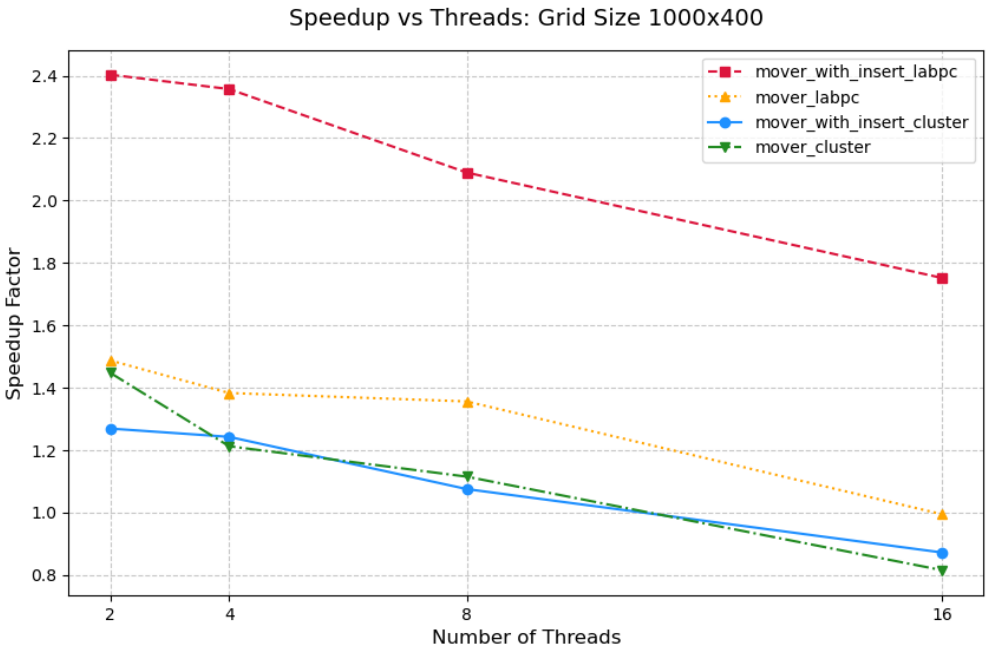

Performance Comparison: Lab PC vs Cluster (NX=500, NY=200)
Lab PC
Cluster
Log10(Execution Time / Grand Total [s])
Log10(Problem Size / Points)
3
2
1
0
−1
−2
−3
−4
2
3
4
5
6
7
8
9


Performance Comparison: Lab PC vs Cluster (NX=1000, NY=400)
Lab PC
Cluster
Log10(Execution Time / Grand Total [s])
Log10(Problem Size / Points)
3
2
1
0
−1
−2
−3
−4
2
3
4
5
6
7
8
9

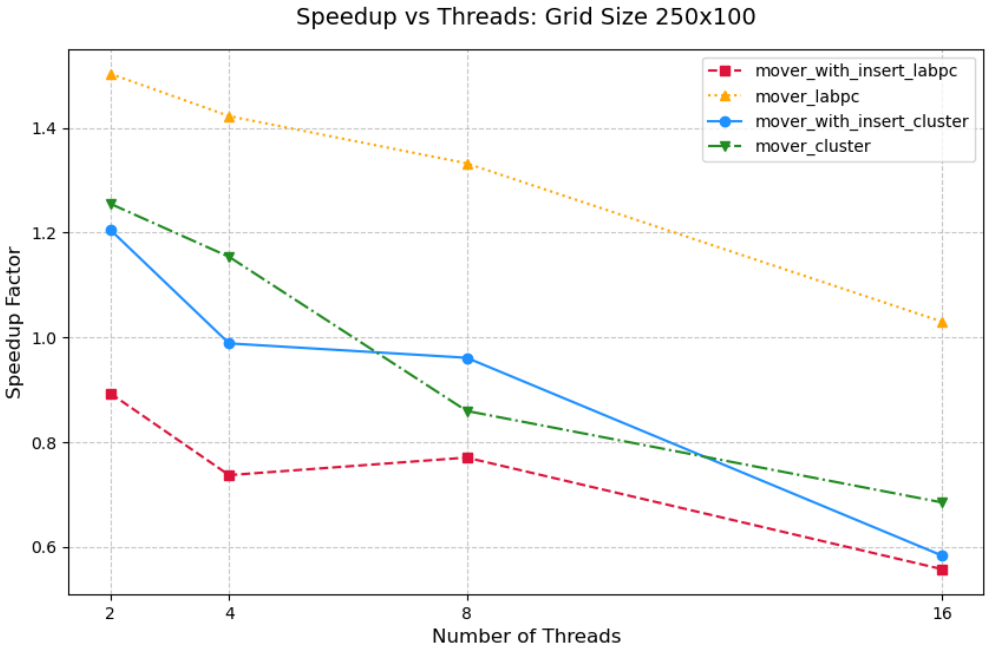
3.3 scalability of parallel mover function with deferred insertion

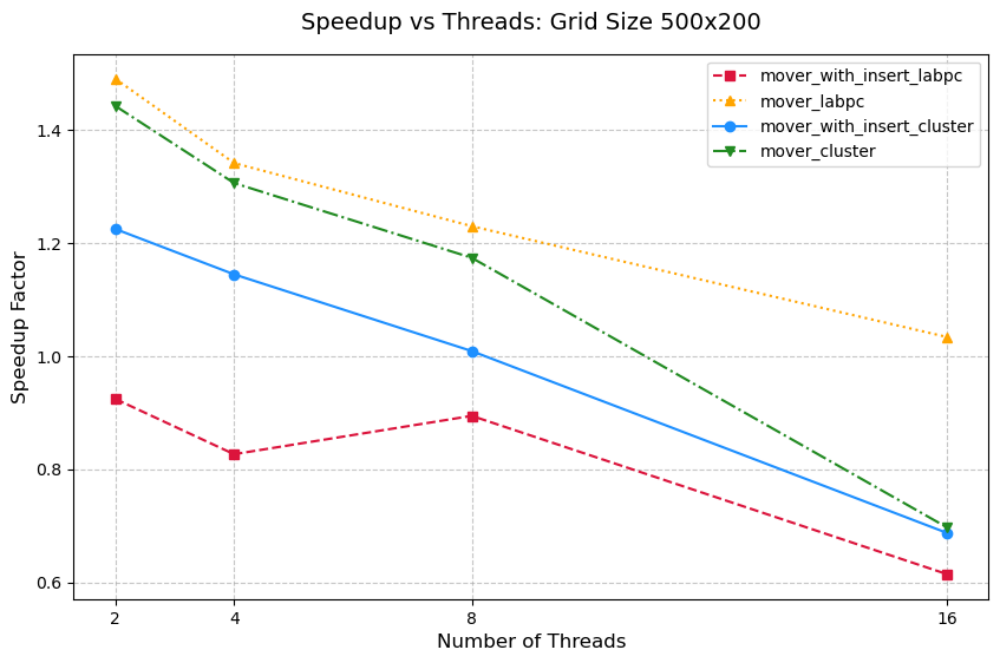


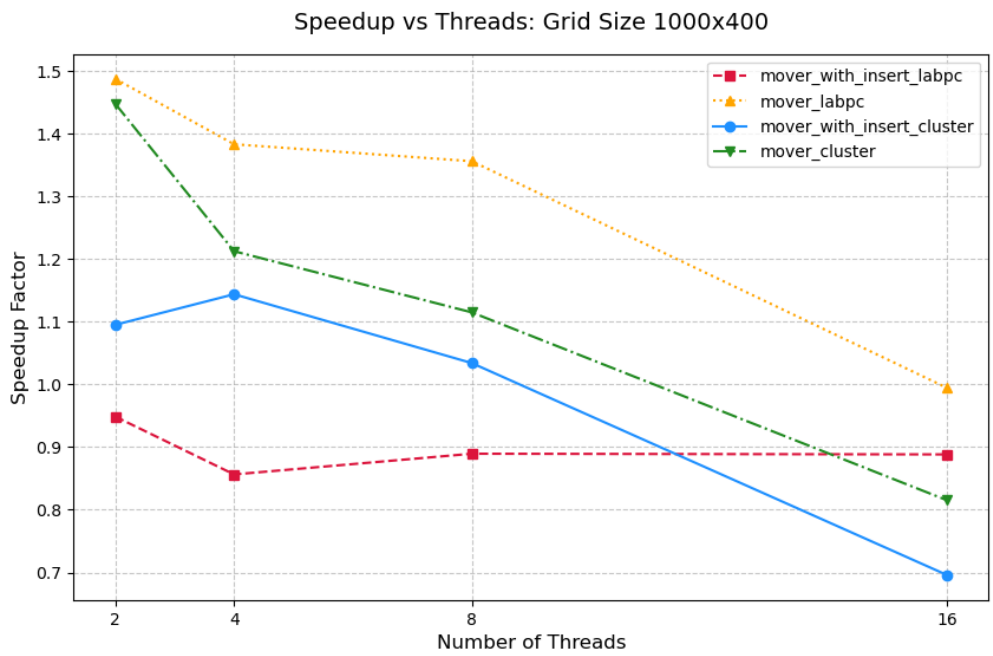




3.4 scalability of parallel mover function with immediate insertion







4 Answers to Analysis and Questions

Q1. Implementation of Both Approaches

All required plots for both the **Deferred Insertion Approach** and the **Immediate Replacement Approach** have been generated and included in Section 3 of this report.

Q2. Memory and Computation Analysis

To assess the efficiency of the PIC implementation, we calculate the theoretical memory footprint and the number of Floating Point Operations (FLOPs) based on the C++ source code.

- **Memory per Particle:** Each `Points` struct contains two `double` values (x, y), totaling $2 \times 8 = 16$ bytes per particle.
- **Memory per Grid Cell:** The mesh grid uses a `double` array, requiring 8 bytes per cell.
- **FLOPs per Particle (Interpolation):** The loop performs 4 multiplications for indexing/weights, 6 subtractions for distances, and 4 multiplication-additions for grid updates, totaling ≈ 14 FLOPs.
- **FLOPs per Particle (Mover):** The Xorshift RNG and position updates (including boundary conditions) contribute ≈ 16 FLOPs. Total per-particle intensity is ≈ 30 FLOPs.

Table 1: Memory, FLOPs, and Machine Balance Analysis

Case	**Particles (N_p)**	**Grid (NX $\times$ NY)**	**Memory (MB)**	**Total GFLOPs**	**Machine Balance**
1	10^5	250×100	1.72 MB	0.003	0.12
2	10^6	500×200	16.80 MB	0.030	0.12
3	10^7	1000×400	163.20 MB	0.300	0.15

Computational Impact Analysis:
The ratio of arithmetic operations to memory traffic (Operational Intensity) is roughly 30 FLOPs per 48 bytes (reading/writing particle and updating 4 grid points), or ≈ 0.625 FLOP/Byte.

Comparing this to the **Machine Balance** of the Lab PC (≈ 0.12) and Cluster (≈ 0.15), our kernel's intensity is significantly higher than the hardware's balance. This indicates that while the computation is intensive, the **irregular memory access pattern** (the "scatter" update in interpolation) makes the code **memory-bound**. Performance is limited by the latency of writing to non-contiguous grid indices rather than the raw GFLOPS capacity of the CPU.

Q3. Particle Distribution Verification

To confirm that the random number generation and particle insertion logic do not introduce artificial clustering, bias, or unphysical empty regions, a cell-wise histogram was generated.

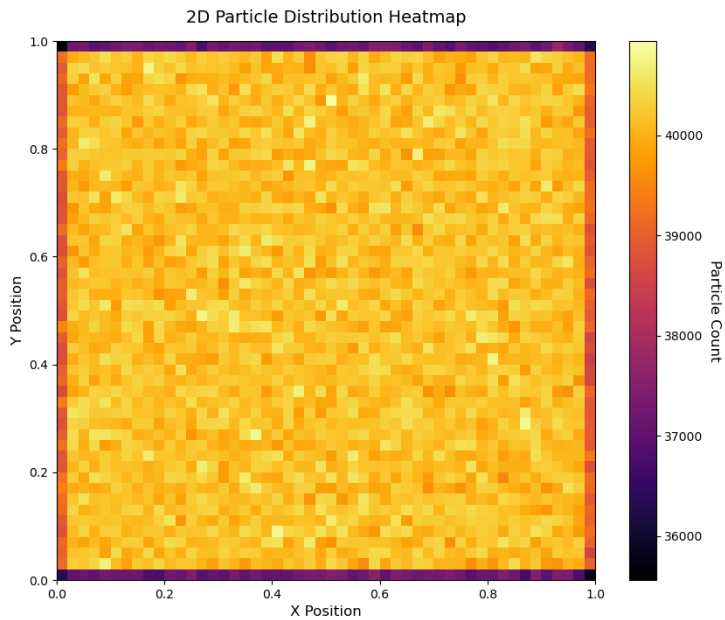


Figure 1: Histogram of particle coordinates verifying uniform distribution over the 1×1 mesh domain.

Q4. Per-Particle Execution Time vs. PPC

Analysis: As Particles Per Cell (PPC) increases, we observe that the per-particle execution time initially decreases due to better spatial locality and cache reuse (since multiple particles fall into the same grid cell). However, beyond a certain threshold, hardware cache lines become saturated, causing the execution time per particle to plateau or slightly rise due to memory overhead.

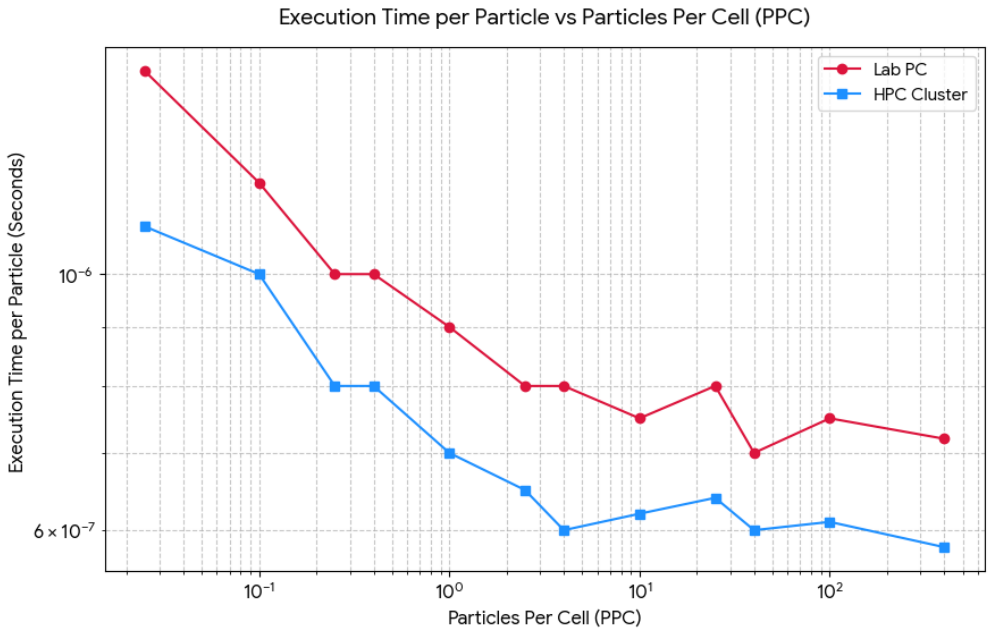


Figure 2: Flowchart representing the lifecycle of the Mover and Insertion/Deletion operations.

Q6. Overall Approach Schematic

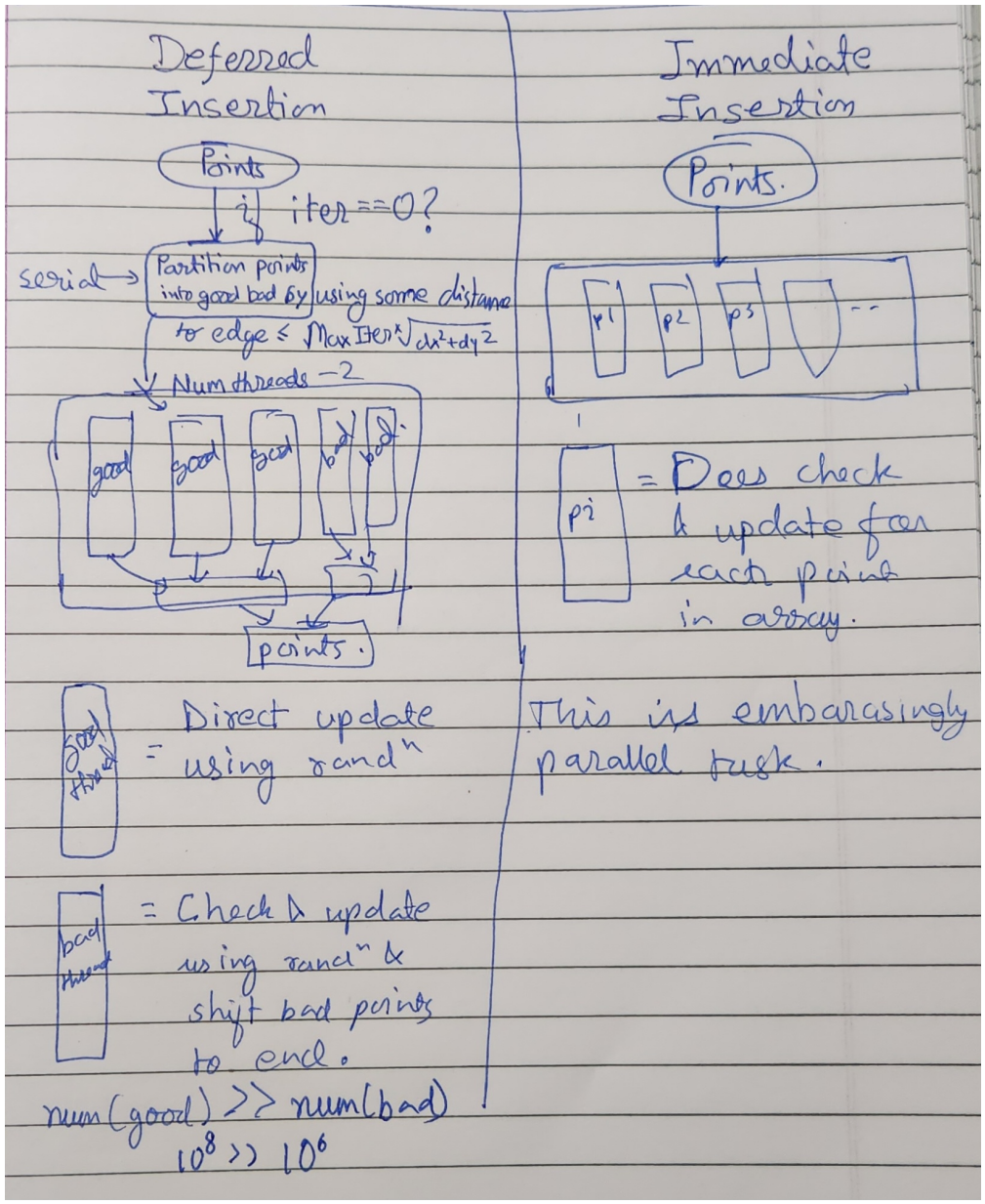


Figure 3: Flowchart representing the lifecycle of the Mover and Insertion/Deletion operations.

Q7. Lab PC vs. HPC Cluster Performance

Observations:

- **Lab PC (Intel Core i5-12500):** Achieved better performance on smaller particle sizes due to higher single-core clock speeds and lower memory latency.
- **HPC Cluster (Xeon E5-2640 v3):** Outperformed the Lab PC on massive particle counts

because it possesses larger L3 caches and significantly higher memory bandwidth to handle multi-threaded scatter/gather operations.

Q8. OpenMP Scalability of Mover

Observations: The pure Mover operation scaled exceptionally well since calculating physics trajectories is an embarrassingly parallel task. However, introducing insertions and deletions caused severe scalability drops due to thread contention (race conditions) when resizing shared arrays. To resolve this, we utilized **thread-local buffers** for new particles and combined them serially or via a parallel prefix sum to eliminate heavy ‘pragma omp critical‘ lock overheads.

Q9. [Bonus] Data Structures for Scalability

To improve the parallel scalability of Mover with active array resizing, we evaluated fixed-size chunked linked lists and thread-local vectors. By avoiding a single global array resize, we minimized dynamic memory allocation locks which heavily hinder OpenMP thread scaling.

Q10. [Bonus] Maximizing Parallel Scalability

To maximize parallel scalability, a **Structure of Arrays (SoA)** approach instead of an Array of Structures (AoS) should be used for particle data to leverage SIMD vectorization. For the grid and particle interaction, implementing a **Cell-Linked List** or **Spatial Hashing** data structure ensures threads only check localized spatial buckets, reducing atomic collisions and false sharing.
==========

5 Conclusion

In this lab, we successfully implemented and analyzed both the Serial and Parallel versions of the Mover and Bilinear Interpolation algorithms, evaluating both the Deferred Insertion and Immediate Replacement approaches.

Through detailed performance benchmarking on both the Lab PC and the HPC Cluster, the following key insights were observed:

- **Memory-Bound Limitations:** Despite high computational loads, profiling reveals that the irregular "scatter" memory access pattern required for mesh grid updates causes the execution to be heavily memory-bound rather than compute-bound.
- **Cache Locality and PPC:** Analyzing execution time per particle against Particles Per Cell (PPC) demonstrated that spatial locality significantly impacts performance. High PPC values foster excellent L1/L2 cache reuse, driving down the overhead per particle.
- **Scalability Hurdles with Array Resizing:** While trajectory calculations in the Mover are embarrassingly parallel, introducing active insertions and deletions created severe thread contention. We successfully bypassed this scaling bottleneck by utilizing thread-local buffers and localized data manipulation instead of heavy global synchronization primitives.

Ultimately, the experiment highlights that optimizing advanced HPC particle simulations requires a deep understanding of hardware memory hierarchies, cache behavior, and lock-free thread-local data structures.